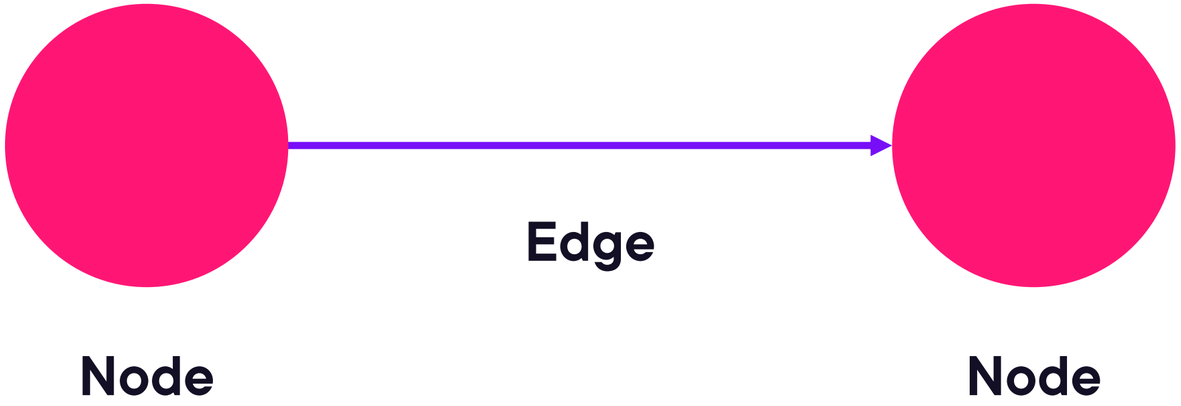
Basics of LangGraph Workflows

Creating Your First LangGraph Workflow

Craig Golightly
Principal Generative AI Author, Pluralsight

Graph



LangGraph Application

LangGraph CLI

Run application locally

LangGraph Studio

Web interface for application

Python >= 3.11

```
$ python --version
Python 3.11.11
```

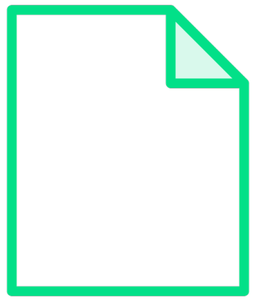
```
$ python -m venv venv
$ source venv/bin/activate
(venv) $
```

```
(venv) $ pip install "langgraph-cli[inmem]"
```

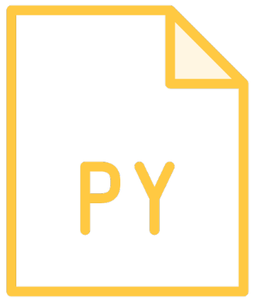
Install LangGraph CLI

Python >= 3.11
Use a virtual environment
Install LangGraph CLI with inmem extra

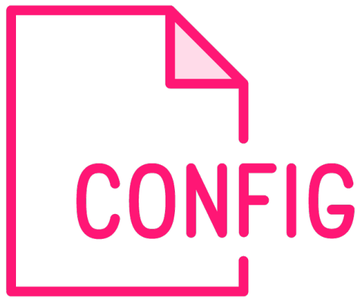
Basic LangGraph Application



Dependency file



Python file



LangGraph API configuration file

Dependency File

(pip)

requirements.txt

```
langgraph
```

LangGraph API Configuration File

langgraph.json

```
{
  "graphs": {
    "basic_graph": "./basic.py:graph"
  },
  "dependencies": [
    "."
  ]
}
```

Python for LangGraph Workflow

State objects

Track data

Node functions

Logic

Graph construction

Nodes and edges

```python
from typing_extensions import (
  TypedDict, Optional
)

class FlightState(TypedDict):
  name: str
  flight_number: str
  upgrade: Optional[bool]
  message: Optional[str]
```

- Basic state object
- Track inputs as strings
- Optional items to track during workflow

Node Functions

```
check_in

offer_upgrade

confirm_check_in
```

```
def node_function(state: FlightState) -> FlightState:
```

Check In

```
def check_in(state: FlightState) -> FlightState:
  state["message"] = (
    f"Checking in {state['name']} "
    f"on flight {state['flight_number']}."
  )
  return state
```

Offer Upgrade

```
def offer_upgrade(state: FlightState) -> FlightState:
  state["message"] = (
    f"Upgrade confirmed for {state['name']} ",
    f"on flight {state['flight_number']}! Safe travels."
  )
  return state
```

Confirm Check In

```
def confirm_check_in(state: FlightState) -> FlightState:
  state["message"] = (
    f"Check-in confirmed for {state['name']} ",
    f"on flight {state['flight_number']}! Safe travels."
  )
  return state
```

Check Upgrade

```python
import random
from typing import Literal

def check_upgrade(
  state: FlightState
) -> Literal["confirm_check_in", "offer_upgrade"]:
  # simulate looking up status from a datasource
  if random.random() < 0.5:
    state["upgrade"] = True
    return "offer_upgrade"

  state["upgrade"] = False
  return "confirm_check_in"
```

Build Graph Workflow

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(FlightState)
builder.add_node("check_in", check_in)
builder.add_node("offer_upgrade", offer_upgrade)
builder.add_node("confirm_check_in", confirm_check_in)

builder.add_edge(START, "check_in")
builder.add_conditional_edges("check_in", check_upgrade)
builder.add_edge("offer_upgrade", END)
builder.add_edge("confirm_check_in", END)

graph = builder.compile()
```

Compiling a Graph

Graph variable must match entry in LangGraph config file

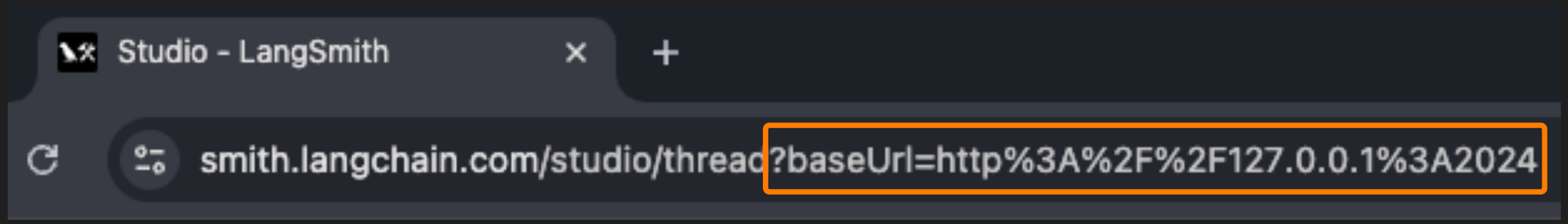
basic.py

```python
graph = builder.compile()
```

langgraph.json

```json
"graphs": {
  "basic_graph": "./basic.py:graph"
}
```

```
(venv) $ langgraph dev
```



Run LangGraph CLI

Starts local instance
LangSmith connects to local instance